

Architecting High-Performance and Integrated Editor Tools in Unity

Executive Summary

The development of custom editor tools within Unity has undergone a significant paradigm shift. Historically dominated by the immediate-mode ImGui system, modern tool creation now demands a more sophisticated approach centered on performance, scalability, and seamless integration. This report provides an exhaustive analysis of best practices for architecting professional-grade Unity editor extensions that are not only powerful but also efficient and intuitive to use. The core thesis is that performance and integration are not features to be added later but are foundational architectural pillars that must be established from the outset. This analysis establishes UI Toolkit as the definitive foundation for building performant and maintainable editor user interfaces. Its retained-mode architecture and features like UI virtualization via the ListView element are critical for handling the large datasets common in modern game development, preventing the lag and unresponsiveness that plague complex ImGui-based tools.

Furthermore, the report details a move away from tightly coupled, monolithic tool designs towards a decoupled, modular architecture facilitated by ScriptableObjects. By leveraging ScriptableObjects as both data containers for settings and as "Event Channels" for communication, developers can create a suite of tools that interact seamlessly without direct, brittle code dependencies. This approach makes the entire toolchain more robust, easier to debug, and significantly more scalable in a team environment.

Finally, the report synthesizes these principles into a practical design for a unified asset toolset, including an advanced Asset Palette with powerful search capabilities and a Prefab Painter. This case study demonstrates the concrete application of metadata tagging, fuzzy search algorithms, and robust data management strategies, including a nuanced approach to Unity's Undo system to ensure data integrity. The recommendations herein are intended to guide technical leads and experienced developers in building the next generation of editor tools that empower creative workflows rather than hindering them.

Part I: The Performance Imperative in Modern Editor Tools

The primary interface between a developer and their work is the editor itself. Tools that lag, freeze, or perform unpredictably introduce friction that directly impacts productivity and morale. Consequently, performance is the most critical feature of any editor extension. This section establishes the foundational principles of performance-driven tool development, focusing on the UI layer and the broader project ecosystem.

Chapter 1: UI Toolkit: The Foundation for Performant Editor UI

The decision to adopt UI Toolkit over the legacy ImGui system is the single most impactful choice for creating modern, scalable, and responsive editor tools. This choice represents a

fundamental architectural shift from a processor-intensive, immediate-mode rendering loop to an efficient, retained-mode, event-driven system.

1.1 The Architectural Shift: From Immediate Mode (ImGui) to Retained Mode (UI Toolkit)

The performance difference between ImGui and UI Toolkit is not incremental; it is architectural. ImGui operates on an "immediate mode" paradigm, where the entire user interface is procedurally drawn from scratch within the `OnGUI()` method every frame. While this offers simplicity for trivial interfaces, its computational cost scales linearly with UI complexity. As a tool grows, adding more elements and logic, the CPU load within `OnGUI()` increases, inevitably leading to a sluggish and unresponsive editor experience.

UI Toolkit, conversely, employs a "retained mode" architecture, a model analogous to modern web and application frameworks. It builds a persistent hierarchy of `VisualElement` objects that is retained in memory. The system then only repaints elements that have changed, a process often referred to as "dirtying" the UI. This event-driven approach is fundamentally more efficient. An action, such as a button click or a data change, triggers a repaint of only the necessary components, rather than the entire window. This decoupling of rendering from the frame rate means that a complex UI Toolkit window has a minimal performance footprint when idle, a stark contrast to an equivalent ImGui window which consumes CPU resources continuously.

For any tool intended for long-term use, feature expansion, and deployment within a professional team, this architectural superiority makes UI Toolkit the only viable choice. To build upon ImGui is to accept an inherent and ultimately unsolvable performance bottleneck that will frustrate and alienate users.

Table 1: UI Toolkit vs. ImGui for Editor Development (2025 Outlook)

Feature	UI Toolkit	ImGui	Architectural Implication
Performance Model	Retained Mode: Event-driven, repaints only on change.	Immediate Mode: Redraws entire UI every frame in <code>OnGUI()</code> .	UI Toolkit offers vastly superior performance and scalability for complex UIs by minimizing redundant processing. ImGui becomes a performance bottleneck as complexity increases.
WYSIWYG Workflow	Yes, via UI Builder. Decouples UI layout from logic (UXML/USS).	No. UI is defined entirely in C# code, mixing layout and logic.	UI Toolkit enables a modern, designer-friendly workflow. It allows UI specialists to build and style interfaces without deep C# knowledge, promoting better team collaboration and faster iteration.

Feature	UI Toolkit	ImGui	Architectural Implication
Styling	USS (CSS-like stylesheets). Centralized, reusable, and powerful styling.	Programmatic GUIStyle objects. Styling is verbose, decentralized, and harder to maintain.	USS allows for the creation of consistent visual themes across an entire tool suite. Changes to a single stylesheet can update dozens of tools, ensuring visual consistency and dramatically reducing maintenance overhead.
Data Binding	Native support for binding UI to SerializedProperty and other data sources.	No native support. Requires manual code to sync data with UI state.	Data binding automates the synchronization of data and UI, reducing boilerplate code, minimizing bugs, and creating a clearer data flow. This is essential for complex, data-driven tools.
Layout System	Flexbox-based. Powerful, flexible, and standard for responsive design.	GUILayout system. Sequential and less flexible for complex, adaptive layouts.	Flexbox allows for the creation of sophisticated, responsive layouts that adapt to window resizing, which is critical for dockable editor windows that can exist in various shapes and sizes.
Asynchronous Support	Excellent. Event-driven nature works well with async operations.	Poor. The OnGUI() loop is synchronous and difficult to integrate with non-blocking tasks.	UI Toolkit's structure naturally supports modern asynchronous programming, allowing long-running tasks to execute without freezing the UI, leading to a more responsive user experience.
Scalability	High. The separation of structure (UXML), style (USS), and logic (C#) is designed for complexity.	Low. The intermingling of logic and layout in a single C# file becomes unmanageable in large tools.	UI Toolkit's architecture is inherently more scalable and maintainable, making it the professional choice for tools that are

Feature	UI Toolkit	ImGui	Architectural Implication
			expected to evolve and be supported over a project's lifecycle.

1.2 Mastering Large Datasets: The ListView and UI Virtualization

For any editor tool that must display a large collection of items—such as the user-queried "Asset Palette"—UI virtualization is not an optimization but an absolute necessity. A naive implementation using a ScrollView would attempt to create a VisualElement for every single item in the dataset at once. In a project with thousands of assets, this would trigger a massive, multi-second editor freeze and an enormous memory allocation, rendering the tool unusable. The ListView element is UI Toolkit's solution to this problem, and it is the cornerstone of performant list-based interfaces. It implements UI virtualization by creating only enough visual elements to fill the visible viewport. As the user scrolls, elements that move out of view are not destroyed; they are placed in a pool and recycled to display new items scrolling into view. This is managed through two key callback functions:

- **makeItem:** A factory function that is called to create the initial pool of visual elements. It is invoked only a handful of times, enough to populate the visible area.
- **bindItem:** This function is called whenever an item needs to be displayed. It takes a recycled visual element from the pool and binds it to the corresponding data item from the source list (e.g., setting an icon and a label).

This mechanism brilliantly decouples the performance of the UI from the size of the underlying dataset. Whether the ListView is bound to 100 assets or 100,000, its memory footprint and rendering performance remain nearly constant, dependent only on the number of visible items. For tools like an asset browser, a level-of-detail manager, or a complex animation event editor, the use of ListView (or its hierarchical cousin, TreeView) is non-negotiable.

1.3 Asynchronous Operations and Maintaining Editor Responsiveness

A performant tool is one that not only executes tasks quickly but also *feels* responsive at all times. Long-running operations, such as searching the entire AssetDatabase, loading complex asset previews, or performing file I/O, must not be executed on the editor's main thread. Doing so will block all UI updates, causing the editor to freeze until the operation is complete—a hallmark of an amateurish and frustrating tool.

To avoid this, heavy logic must be offloaded from the UI thread. Unity's editor environment supports several patterns for this. While standard C# `async/await` and `Task.Run` can be used for background processing, a particularly useful pattern for editor scripting is the `EditorCoroutine`. `EditorCoroutines` allow a long-running task to be broken up and executed over multiple editor frames, yielding control back to the editor between steps to keep the UI responsive.

A typical asynchronous workflow in an Asset Palette search would be:

1. The user types in the search TextField. A callback is triggered.
2. Instead of searching synchronously, the tool starts an `EditorCoroutine` or a background `Task`. A loading spinner or "Searching..." message is displayed in the UI.
3. The coroutine or task executes the `AssetDatabase.FindAssets` query. Because this is not on the main thread, the user can continue to interact with the editor.
4. Once the search is complete, the results are marshaled back to the main thread.

5. The ListView's itemsSource property is updated with the new list of asset GUIDs, and its Rebuild() method is called to refresh the view.

This asynchronous design pattern is essential for creating a professional user experience. It ensures that the tool remains interactive and provides clear feedback to the user, even when performing computationally expensive work in the background. Unity's Background Tasks window can be used to provide users with visible progress for these operations.

Chapter 2: General Editor Performance and Optimization

The performance of an editor tool does not exist in a vacuum. It is deeply intertwined with the overall health of the Unity project it operates within. A tool's internal code can be perfectly optimized, but if it is forced to interact with a poorly managed project, it will still be slow. Therefore, a holistic approach to editor performance must include best practices for the asset pipeline and memory management.

2.1 The Asset Pipeline as a Performance Factor

An unoptimized asset pipeline is a common source of poor editor performance. Tools that browse or manipulate assets, like an Asset Palette, are directly affected by the import settings of those assets. Loading a thumbnail for a 4K texture with compression disabled and Read/Write Enabled checked is orders of magnitude slower and more memory-intensive than loading a preview for a properly configured asset.

A robust tool suite should not only be performant itself but should also encourage or enforce project-wide performance standards. Two key Unity features are essential for this:

- **AssetPostprocessor:** This is an editor script that can hook into the asset import process. It allows developers to automate the enforcement of standards, such as automatically setting texture compression, disabling Read/Write Enabled on meshes, or ensuring audio files use appropriate settings for their type.
- **Presets:** Presets allow developers to save a collection of import settings as an asset. These can then be applied manually or set as defaults for specific folders. For example, a preset can be created for "UI Icons" that ensures any texture placed in the Assets/UI/Icons folder is automatically imported as a Sprite with mipmaps disabled.

By leveraging these tools, developers can ensure that the assets a custom tool interacts with are already optimized, which is a prerequisite for the tool itself to be performant. A mature toolset might even include an "Asset Auditor" tool that scans the project for assets that violate these standards and offers to fix them automatically.

2.2 Memory Management in the Editor Context

The Unity Editor is a long-running application, and custom tools are guests within its process. Poor memory management in an editor extension—such as memory leaks or excessive garbage generation—has a cumulative, degrading effect on the entire editor session, not just the tool itself. Over time, this can lead to a sluggish, unstable editor that requires frequent restarts.

Developers must apply the same memory management discipline to editor code as they do to runtime code:

- **Control Memory Footprint:** The Read/Write Enabled option on textures and meshes is a primary culprit for memory bloat. When enabled, it creates a duplicate of the asset data in

CPU-addressable memory, effectively doubling its footprint. This should be disabled unless the script explicitly needs to modify the asset data at runtime.

- **Pool Reusable Objects:** Just as in runtime UI, frequently created and destroyed UI elements should be pooled to avoid churning memory and triggering the garbage collector (GC). The ListView handles this automatically for its items, but for other recurring custom elements, manual object pooling is a crucial practice.
- **Unload Unused Assets:** When a tool is finished with an asset it has loaded explicitly (e.g., a large model for a preview window), it should unload it using `Resources.UnloadUnusedAssets()` or a more targeted approach to free up memory.
- **Profile Your Tools:** The Unity Profiler and, more specifically, the Memory Profiler package are indispensable tools. They can connect to the editor itself and should be used to hunt down memory leaks, identify sources of GC allocation, and ensure that a tool cleans up after itself properly when closed.

Part II: Architectural Patterns for Integrated and Maintainable Tools

To create a suite of tools that work together seamlessly, as requested by the user, a modern architecture that prioritizes decoupling and maintainability is required. This section moves beyond direct, brittle references between tool components and explores robust, message-based systems that form the communication backbone of a professional toolchain.

Chapter 3: Decoupling with Event-Driven Architecture

An event-driven architecture is the key to making tools "talk to each other" without being rigidly dependent on one another. This allows for a modular system where new tools can be added or existing ones modified with minimal impact on the rest of the suite.

3.1 The Observer Pattern: A Foundation for Communication

The Observer design pattern provides the fundamental model for this architecture. It defines a one-to-many dependency between objects where a "publisher" (or subject) maintains a list of "observers" (or subscribers) and notifies them automatically of any state changes. The crucial aspect of this pattern is the decoupling it provides: the publisher has no knowledge of the concrete classes of its observers, only that they implement a subscriber interface.

In the context of the user's request, this pattern is a perfect fit:

- **Publisher:** The **Unified Asset Palette** window. When a user selects a prefab, it raises an `AssetSelected` event, passing the selected `GameObject` as a payload.
- **Observer:** The **Prefab Painter** window. It subscribes to the `AssetSelected` event. When the event is raised, its event handler method is invoked, and it updates its internal state to use the new prefab as its active "brush."

This ensures the Asset Palette can function entirely on its own, without a single line of code that mentions the Prefab Painter. Similarly, the Prefab Painter can receive its active asset from any source that raises the correct event. This modularity is essential for building a scalable and maintainable toolset.

3.2 ScriptableObject Architecture: The Modern Approach to Editor State and

Events

While the Observer pattern can be implemented with standard C# events (e.g., public static event Action<GameObject> OnAssetSelected;), a more powerful and Unity-native approach is to use ScriptableObjects as the communication medium. This pattern, often referred to as ScriptableObject Architecture (SOA), has two key components: ScriptableObject Variables and ScriptableObject Events (or "Event Channels").

Using ScriptableObjects as **Event Channels** provides a tangible, asset-based alternative to abstract static C# events. Instead of a static event, an "Event Channel" is a ScriptableObject asset that contains a UnityAction delegate and a public RaiseEvent() method.

The workflow becomes:

1. Create a ScriptableObject asset in the project, for example, SelectedPrefabChannel.asset.
2. The Asset Palette EditorWindow has a public field that holds a reference to this SelectedPrefabChannel asset. When a user selects a prefab, the Palette calls _selectedPrefabChannel.RaiseEvent(selectedPrefab).
3. The Prefab Painter EditorWindow also has a public field referencing the *exact same* SelectedPrefabChannel.asset. In its OnEnable method, it subscribes a handler method to the channel's OnEventRaised delegate. In OnDisable, it unsubscribes.

This approach transforms the abstract connection between tools into a visible, inspectable, and configurable project asset. A developer or technical designer can see in the Inspector that both tools are "plugged into" the same channel. They can even swap this asset for another one to change the tool's behavior without recompiling code. This makes the entire communication layer transparent, easier to debug (a custom inspector for the channel can log all events raised), and far more flexible than hard-coded static event systems.

Table 2: Inter-Tool Communication Patterns

Pattern	Implementation	Pros	Cons	Best Use Case
Direct Reference	EditorWindow.GetWindow<OtherTool>().DoSomething();	Simple for trivial interactions.	Creates extremely tight coupling. Fails if the other window is not open. Breaks easily during refactoring.	Not recommended for any professional tool suite. Suitable only for quick, temporary prototypes.
Static C# Events	public static event Action<T> OnSomethingHappened;	Decouples publisher from subscriber. Performant.	Connection is invisible in the editor. Can lead to memory leaks if subscribers don't unregister. Harder for non-programmers to understand or modify.	Core systems programming where inspector visibility is not required and performance is paramount.
Singleton / Service Locator	ToolManager.Instance.Register(this); ToolManager.Instance	Provides a central point of access.	Introduces global state, making unit testing difficult.	Managing truly global editor state or services, but

Pattern	Implementation	Pros	Cons	Best Use Case
	nce.Notify(...);		Hides dependencies. Can become a monolithic bottleneck.	should be used with extreme caution. Often an anti-pattern.
ScriptableObject Event Channels	Tools reference a shared ScriptableObject asset and use its delegates for communication.	Highly Decoupled. Visible & Inspectable: The connection is a project asset. Designer-Friendly: Can be configured in the Inspector. Persistent.	Slightly more setup than static events. Can lead to a large number of assets if overused.	The recommended pattern for all tool-to-tool communication. Ideal for creating modular, scalable, and maintainable editor tool suites.

Chapter 4: Robust Data Management and State Persistence

Professional tools must reliably manage their data and state. This includes persisting user settings across editor sessions and providing a non-negotiable safety net for user actions through a robust Undo/Redo system.

4.1 Leveraging ScriptableObjects for Tool Settings and Shared Data

Just as ScriptableObjects excel as event channels, they are also the ideal solution for managing a tool's settings and shared state. Storing configuration data (e.g., the Prefab Painter's brush size, the Asset Palette's last used filters) directly within an EditorWindow script is fragile, as the window instance can be destroyed and recreated. Using EditorPrefs is also suboptimal, as it stores data locally on a user's machine, making it impossible to share or version control. By creating a ToolSettings.asset using a custom ScriptableObject class, the tool's configuration becomes a first-class citizen of the Unity project. This approach has profound benefits for a development team:

- **Persistence & Version Control:** The settings asset is saved with the project and can be tracked in source control systems like Git.
- **Shareability:** A new team member automatically gets the standard tool configuration when they clone the project.
- **Presets:** Multiple settings assets can be created (e.g., LevelDesignSettings.asset, VFXArtistSettings.asset). Users can swap between these presets to configure a tool for different tasks, all without code changes.
- **Decoupling:** The state of the tool is decoupled from the UI (EditorWindow), adhering to the principle of separation of concerns. Multiple tools can even share and react to the same settings asset.

This elevates tool configurations from opaque, user-specific data into a transparent, collaborative part of the project's ecosystem, making the entire toolset more powerful and manageable.

4.2 Implementing a Reliable Undo/Redo System

A professional editor tool must never perform a destructive action without providing the user a way to undo it. Unity's Undo class is the primary API for this functionality. The most common method, `Undo.RecordObject(objectToModify, "Action Description")`, must be called *before* any properties on the target object are changed. This registers the object's current state, allowing the system to revert to it. For creating and destroying objects, dedicated methods like `Undo.RegisterCreatedObjectUndo` and `Undo.DestroyObjectImmediate` must be used. However, there is a critical and often overlooked nuance when dealing with collections. `Undo.RecordObject` works by creating a *diff* of the object's properties. For complex operations on a List or array, such as removing multiple elements or reordering them within a single user action (like a mouse drag), this diffing mechanism can be unreliable and lead to data corruption upon undoing.

For these specific cases, the correct and safe method to use is `Undo.RegisterCompleteObjectUndo`. This method forgoes diffing and instead saves a complete snapshot of the object's state onto the undo stack. While this is more memory-intensive, it guarantees that the undo operation will restore the object to its exact previous state, preserving data integrity. Therefore, a robust tool must analyze its data modification patterns:

- For simple, single-property changes: `Undo.RecordObject` is efficient and sufficient.
- For any complex or holistic manipulation of a collection:
 `Undo.RegisterCompleteObjectUndo` is the only safe and reliable choice.

Ignoring this distinction can lead to subtle, hard-to-reproduce bugs that corrupt user data—a catastrophic failure for any professional tool.

Part III: Design and Enhancement of a Unified Toolset

This final section serves as a practical case study, applying the architectural principles and performance best practices discussed previously to design the user's requested integrated tool suite: a Unified Asset Palette and a Prefab Painter.

Chapter 5: The Unified Asset Palette: A Case Study in Advanced Tool Design

The Asset Palette is the central hub of the proposed toolset. It must be fast, powerful, and intuitive, serving as the primary interface for asset discovery and selection.

5.1 Core Architecture: Building a Central, Data-Driven Hub

The palette will be constructed as an `EditorWindow` following modern best practices:

- **UI (View):** The entire UI will be built with UI Toolkit. The structure will be defined in a UXML file (`AssetPalette.uxml`) and styled with a USS stylesheet (`AssetPalette.uss`). The main display for assets will be a `ListView` to ensure high performance with thousands of items. A `TextField` will serve as the primary search input, and various `Toggle` or `DropDownField` elements will control filters.
- **Data (Model):** All persistent state, such as the current search query, active filters, and display settings, will be stored in a `ScriptableObject` asset, `AssetPaletteSettings.asset`. This ensures the palette's state is saved between editor sessions and can be version

controlled.

- **Communication (Controller/Events):** When an asset is selected in the ListView, the palette will not directly communicate with any other tool. Instead, it will raise an event on a `GameObjectEventChannelSO` asset named `SelectedAssetChannel.asset`, broadcasting the selected asset to any listening tool.

5.2 Advanced Search Functionality

To be more powerful than Unity's default Project window, the Asset Palette must offer advanced search and filtering capabilities.

- **Type, Label, and Path Filtering:** The foundation of the search will be Unity's `AssetDatabase.FindAssets` method. This powerful API accepts a filter string that can search by type (t:prefab), label (l:character), name, and folder path. The UI's filter toggles and dropdowns will programmatically construct this filter string to execute precise, efficient, and native searches.
- **Custom Metadata Tagging:** While Unity's primary metadata system is the .meta file for import settings, true custom metadata (e.g., "style: stylized", "environment: forest") is not natively supported on assets in the same way. The most straightforward approach is to leverage Unity's built-in **Labels**. The tool can provide a UI for managing and assigning these labels to assets. For more complex needs, mimicking the functionality of Unity Asset Manager, a custom system can be built. This could involve a central `ScriptableObject` acting as a database that maps asset GUIDs to a dictionary of custom key-value tags. The search function would then query this custom database in addition to the `AssetDatabase`.
- **Fuzzy Search Implementation:** To provide a more forgiving and intelligent search experience, an approximate string matching algorithm is essential. After retrieving an initial set of results from `AssetDatabase.FindAssets`, a secondary filtering pass can be performed using `UnityEditor.Search.FuzzySearch.FuzzyMatch`. This method compares the user's query against each asset name and returns a score indicating the quality of the match. The results can then be sorted by this score, pushing the most relevant items to the top. Furthermore, the `FuzzyMatch` method provides the indices of the matched characters in the string. This information should be used to highlight the matched characters within the asset's label in the ListView, providing immediate visual feedback to the user and clarifying why a particular result was returned.

5.3 UI/UX Principles for Developer-Centric Tools

The design of the Asset Palette's UI must respect the workflow and context of a developer.

- **Immediate Context:** All core functionality—search input, filter controls, and the results list—must be contained within the single, unified panel. The user should be able to perform a complete search and selection workflow without opening any additional popups or windows.
- **Intermediate Context:** The tool must be designed as a companion to other windows, particularly the Scene and Hierarchy views. It should be dockable and function well in both wide and narrow layouts. Selecting an asset should update the `SelectedAssetChannel` but should not necessarily steal keyboard focus from other windows, allowing a developer to, for example, select a prefab in the palette and immediately use a hotkey in the Scene view.

- **Broad Context:** The UI should be information-dense and avoid wasting screen real estate with excessive padding or large decorative elements. Developers often work with multiple tools and windows simultaneously, and a well-designed tool respects this by being compact and efficient in its presentation of information.

Chapter 6: Integrating the Tool Suite: A Practical Workflow

This chapter demonstrates the power of the decoupled architecture by connecting the Asset Palette to a Prefab Painter tool.

6.1 Connecting the Asset Palette to a "Prefab Painter"

The integration is achieved not through direct code dependencies, but by having both tools reference the same ScriptableObject Event Channel.

Step-by-Step Implementation:

1. An EditorWindow script for the PrefabPainter is created. It contains logic in OnSceneGUI to handle mouse input and place prefabs.
2. A public field is added to the PrefabPainter: `public GameObjectEventChannelSO ActiveAssetChannel;`. In the editor, the same `SelectedAssetChannel.asset` used by the Asset Palette is assigned to this field.
3. A private field, `private GameObject _currentPrefabToPaint;`, is added to store the active "brush."
4. A handler method, `private void OnActiveAssetChanged(GameObject newAsset)`, is created. This method simply sets `_currentPrefabToPaint = newAsset;`.
5. In the PrefabPainter's `OnEnable()` method, the handler is subscribed to the event channel: `ActiveAssetChannel.OnEventRaised += OnActiveAssetChanged;`
6. Crucially, in the `OnDisable()` method, the handler is unsubscribed to prevent memory leaks: `ActiveAssetChannel.OnEventRaised -= OnActiveAssetChanged;`

The result of this architecture is a seamless and robust workflow. When a user clicks an asset in the Asset Palette, it raises an event on the `SelectedAssetChannel`. The Prefab Painter, listening to this channel, receives the event and updates its active prefab instantly. The two tools are perfectly synchronized yet remain completely unaware of each other's existence, a hallmark of a well-designed, modular system.

6.2 Ensuring Seamless Scene and Hierarchy Interaction

A tool's integration extends to Unity's native windows. When the Prefab Painter instantiates a new object in the scene, it must behave like a native editor action.

1. **Object Creation:** The new prefab instance is created. Immediately after, `Undo.RegisterCreatedObjectUndo(newInstance, "Paint Prefab");` is called to make the action undoable.
2. **Selection:** To provide immediate feedback, the newly created object should be selected in the Hierarchy. This is accomplished by setting `Selection.activeGameObject = newInstance;`
3. **Focus:** To draw the user's attention, `EditorGUIUtility.PingObject(newInstance);` can be called, which briefly highlights the object in the Hierarchy window.

Conversely, tools can also listen to changes in the editor state. An EditorWindow can subscribe to the `Selection.selectionChanged` delegate. This would allow a tool, for example, to display

information about the currently selected object in the Hierarchy, creating a bi-directional, context-aware workflow.

Conclusions and Recommendations

The development of professional-grade editor tools in Unity requires a deliberate and modern architectural approach. The era of simple, ImGui-based scripts is insufficient for the complexity and performance demands of today's development workflows.

The following core principles should be adopted for all new editor tool development:

1. **Embrace UI Toolkit as the Standard:** For any non-trivial editor UI, UI Toolkit is the only viable path forward. Its retained-mode architecture, superior performance, and modern workflow capabilities (UXML, USS, UI Builder) are essential for creating scalable and responsive tools. For any feature involving large datasets, the ListView and its UI virtualization capabilities are not optional; they are mandatory for preventing editor freezes.
2. **Architect for Decoupling with ScriptableObjects:** The use of ScriptableObjects as event channels and for data/settings storage should be the default architectural pattern for any tool suite. This approach transforms abstract connections into tangible, inspectable, and version-controllable project assets, resulting in a system that is profoundly more modular, maintainable, and team-friendly than alternatives like static events or singletons.
3. **Prioritize Performance and Responsiveness Holistically:** A tool's performance is a product of its own code and the health of the project it runs in. Best practices must include both optimizing the tool's internal logic with asynchronous operations to maintain UI responsiveness and enforcing project-wide standards for asset configuration. Memory management must be treated with the same rigor as in runtime code to ensure the long-term stability of the editor session.
4. **Design for the Developer's Workflow:** Effective tools are designed with a deep understanding of their user's context. By considering the immediate, intermediate, and broad contexts of a developer's tasks, tools can be made more intuitive, less intrusive, and more efficient, ultimately accelerating the creative process.

By adhering to these principles, development teams can build powerful, integrated, and high-performance in-house tools that become a competitive advantage, enabling faster iteration, better collaboration, and higher-quality end products.

Works cited

1. Why I chose UI Toolkit over ImGui to make my first Unity tool with | by Alireza Forghani Toosi, <https://alirezaft98.medium.com/why-i-chose-ui-toolkit-over-ImGui-to-make-my-first-unity-tool-with-62e70b34e098>
2. Comparison of UI systems in Unity - Unity - Manual, <https://docs.unity3d.com/2020.1/Documentation/Manual/UI-system-compare.html>
3. Manual: Comparison of UI systems in Unity, <https://docs.unity.cn/2021.1/Documentation/Manual/UI-system-compare.html>
4. Unity UI Best Practices for Performance - Wayline, <https://www.wayline.io/blog/unity-ui-best-practices-for-performance>
5. ListView - Unity - Manual, <https://docs.unity3d.com/6000.1/Documentation/Manual/UIE-uxml-element-ListView.html>
6. Best practices for managing elements - Unity - Manual, <https://docs.unity3d.com/6000.1/Documentation/Manual/UIE-best-practices-for-managing-elements.html>
7. List View - Unity | Editor Foundations Design System,

<https://www.foundations.unity.com/components/list-view> 8. [needle-mirror/com.unity.editorcoroutines](https://github.com/needle-mirror/com.unity.editorcoroutines) - GitHub, <https://github.com/needle-mirror/com.unity.editorcoroutines> 9. Namespace `Unity.EditorCoroutines.Editor` | Editor Coroutines | 1.0.0, <https://docs.unity.cn/Packages/com.unity.editorcoroutines@1.0/api/Unity.EditorCoroutines.Editor.html> 10. Class `EditorCoroutine` | Editor Coroutines | 1.0.0, <https://docs.unity.cn/Packages/com.unity.editorcoroutines@1.0/api/Unity.EditorCoroutines.Editor.EditorCoroutine.html> 11. The Background Tasks window - Unity - Manual, <https://docs.unity3d.com/2020.1/Documentation/Manual/BackgroundTasksWindow.html> 12. Configuring your Unity project for stronger performance, <https://unity.com/how-to/project-configuration-and-assets> 13. Optimize performance and quality | Unity's profiling tools, <https://unity.com/features/profiling> 14. Unity UI Optimization Workflow: Step-by-Step full guide for everyone : [r/unity_tutorials](https://www.reddit.com/r/unity_tutorials), https://www.reddit.com/r/unity_tutorials/comments/1bkwebv/unity_ui_optimization_workflow_step_by_step_full/ 15. Your Full Guide to Unity Performance Optimization - Game-Ace, <https://game-ace.com/blog/unity-performance-optimization/> 16. Create modular and maintainable code with the observer pattern ..., <https://learn.unity.com/tutorial/create-modular-and-maintainable-code-with-the-observer-pattern?uv=6&projectId=67bc8deaedbc2a23a7389cab> 17. The Observer Pattern In Unity - Dev Genius, <https://blog.devgenius.io/the-observer-pattern-in-unity-4ee8e12100aa> 18. Improve your Game Architecture with Scriptable Objects - YouTube, <https://www.youtube.com/watch?v=bO8WOHCxPq8> 19. Architect your code for efficient changes and debugging with ... - Unity, <https://unity.com/how-to/architect-game-code-scriptable-objects> 20. Use ScriptableObjects as event channels in game code - Unity, <https://unity.com/how-to/scriptableobjects-event-channels-game-code> 21. Unity Architecture: Scriptable Object Pattern | by Simon Nordon ..., <https://medium.com/@simon.nordon/unity-architecture-scriptable-object-pattern-0a6c25b2d741> 22. Undo - Scripting API - Unity - Manual, <https://docs.unity3d.com/ScriptReference/Undo.html> 23. Scripting API: `Undo.RecordObject` - Unity, <https://docs.unity3d.com/ScriptReference/Undo.RecordObject.html> 24. Please help: Undo for ScriptableObject : [r/Unity3D](https://www.reddit.com/r/Unity3D) - Reddit, https://www.reddit.com/r/Unity3D/comments/3ddoro/please_help_undo_for_scriptableobject/ 25. How can we avoid corrupting lists when using Unity `Undo.RecordObject`? - Stack Overflow, <https://stackoverflow.com/questions/71404886/how-can-we-avoid-corrupting-lists-when-using-unity-undo-recordobject> 26. Getting started with UI toolkit - Unity Learn, <https://learn.unity.com/tutorial/getting-started-with-ui-toolkit> 27. Create a custom control with two attributes - Unity - Manual, <https://docs.unity3d.com/6000.0/Documentation/Manual/UIB-structuring-ui-custom-elements.html> 28. Scripting API: `AssetDatabase.FindAssets` - Unity, <https://docs.unity3d.com/ScriptReference/AssetDatabase.FindAssets.html> 29. Asset Metadata - Unity - Manual, <https://docs.unity.cn/Manual/AssetMetadata.html> 30. Manage custom metadata for assets • Unity Cloud • Unity Docs, <https://docs.unity.com/cloud/asset-manager/manage-custom-metadata> 31. Scripting API: `FuzzySearch` - Unity - Manual, <https://docs.unity3d.com/6000.1/Documentation/ScriptReference/Search.FuzzySearch.html> 32. Scripting API: `Search.FuzzySearch.FuzzyMatch` - Unity, <https://docs.unity3d.com/6000.1/Documentation/ScriptReference/Search.FuzzySearch.FuzzyMa>

tch.html 33. Devs in mind 2025: how to design interfaces for developer tools ...,
<https://evilmartians.com/chronicles/devs-in-mind-how-to-design-interfaces-for-developer-tools>